



第三周

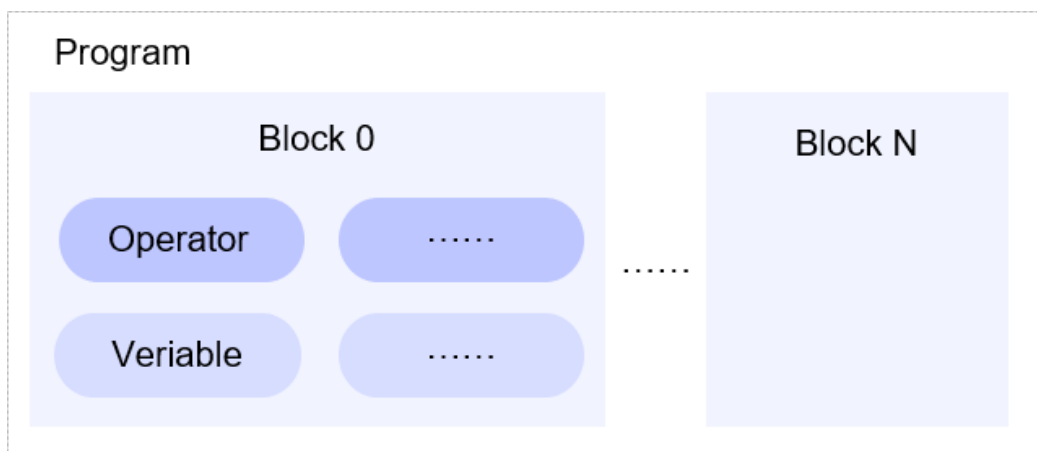
学习

飞桨框架

1. 基于飞桨进行二次开发
 - 飞桨api体系
 - 包含基础API和高层API两部分：基础API丰富全面，成熟完备，提供各种各样计算所需的函数和算子，覆盖深度学习基础、分布式和高性能、工具和调试等方向；高层API（也称为HAPI）是对基础API的再次封装，将模型训练、数据读取等操作封装成简单的API

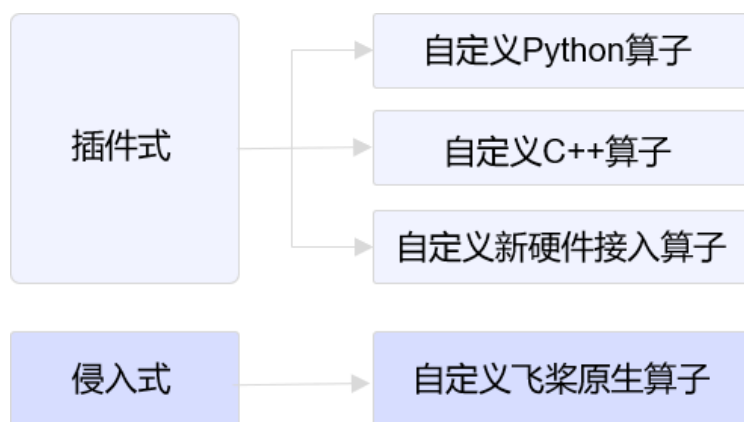


- 自动微分机制
 - 它根据forword()函数自动构建backward()函数。在网络构建时，用户只需要关注前向计算过程，无需进行反向计算的代码开发。
- IR
 - 深度学习框架向上接各种AI应用，向下接各种芯片，为了降低新硬件接入成本，便捷地实现深度学习框架和不同的硬件，如CPU/NPU/GPU之间的交互，飞桨设计了中间表示的概念。IR提出后，每种语言和硬件只需要和IR实现适配即可，大大降低了对接的复杂度。
 - 性能优化，如混合精度、稀疏化加速等策略也是在IR侧执行的。
 - 飞桨采用二层IR的设计，第一层IR也称为Program，表达直观，与程序概念符合，只表达语义逻辑，易于操作；第二层IR也称为Graph，用于执行计算图优化与编译优化，如动态剪枝、显存优化等，提升飞桨框架性能。



- 核心概念

- Tensor
- variable
- Operator
- Block
- Program
- Graph
- Executor
- 第一层IR
 - 构建深度学习模型时，只需定义前向计算网络、损失函数和优化算法，飞桨会自动生成反向传播和梯度优化的流程，该流程由初始化程序（startup_program）与主程序（main_program）实现。
 - 在飞桨中，所有对数据的计算都由Operator表示。为了进一步降低实现难度，在Python端，Operator被封装到paddle.Tensor和paddle.nn等模块中，读者可以使用飞桨提供的API快速完成组网。在组网时，飞桨会不断地向计算图中添加新的Variable和Operator，直到计算图构建完成。启动训练时，飞桨会按照Operator的顺序执行，直到完成训练。
- 第二层IR
 - 第二层IR也称为Graph，用于执行计算图优化与编译优化，提升飞桨框架性能。Graph是一个图表示，其示意如图13所示。借助丰富的图运算和图优化理论，在图表示上可以便捷地进行各种操作，如子图匹配、动态剪枝、死码消除、显存优化等。
 - 用户通过Python语言使用飞桨进行模型构建和训练，但编译优化和推理部署的执行均为C++程序，这使得飞桨兼具轻松的编程体验和极高的执行效率。
- 算子二次研发
 - 飞桨框架支持插件式和侵入式两种二次研发方式



- 插件式

- 自定义的算子直接插入到当前代码中即可使用，无需重新编译飞桨框架
- **自定义C++算子**：使用C++(或CUDA)实现自定义算子，不涉及框架内部概念，无需重新编译飞桨框架，以外接模块的方式使用的算子，需要读者有一定的C++或CUDA基础，但具有性能优势。

•

C++自定义算子实现主要分为两个步骤：1) 使用C++实现 `forward` 和 `backward` 函数；2) 将实现的 `forward` 和 `backward` 函数注册到飞桨。其中C++算子运算函数有特定的写法要求，读者在编码过程中需要遵守，格式如下：

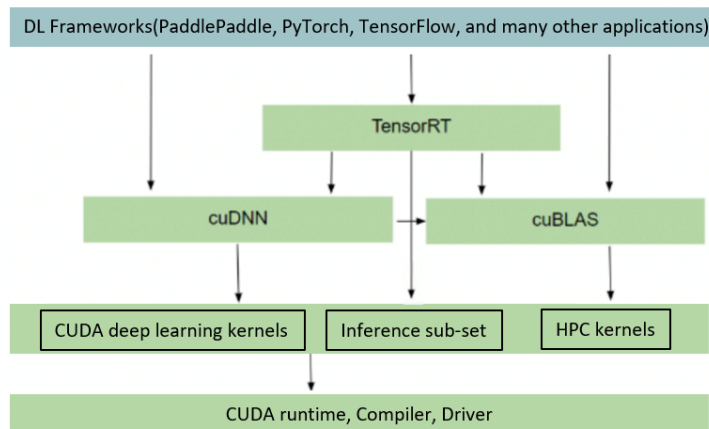
```
std::vector<paddle::Tensor> OpFunction(const paddle::Tensor&x, ... , int attr, ...)  
{  
    ...  
}
```

◦ 侵入式

- 包括自定义飞桨原生算子。操作相对复杂，新增算子需要与飞桨框架一起编译后使用

2. 部署

- AI模型部署主要可以分为服务器侧部署和边缘端侧部署两大类。
 - **高性能的数据中心场景**：性能强大，追求高精度模型和高并发、实时响应的服务质量。
 - 模型融入业务系统：业务系统本身是一个服务架构，某个环节依赖模型实现，可以使用Paddle Inference完成部署。
 - 模型独立作为服务：模型的预测服务可以直接被其它模块在线调用，可以使用Paddle Serving完成部署。
 - **端侧场景**：模型大小和性能受限于硬件能力，需要使用更轻量的网络结构并进行模型压缩。
 - **移动端场景**
 - 模型部署在APP中：可以使用Paddle Lite完成部署。
 - 模型部署在H5小程序中：可以使用更轻量级的Paddle.js完成部署。
 - **物联网场景**：如工控机、工业相机等处于生产线和智能设备，可以使用Paddle Lite完成模型部署。
- 英伟达软件栈



3. Tensor学习

a. 概念

- i. Tensor 可以理解为多维数组，类似于 Numpy 数组（ndarray）的概念。与 Numpy 数组相比，Tensor 除了支持运行在 CPU 上，还支持运行在 GPU 及各种 AI 芯片上，以实现计算加速
- ii. 在飞桨框架中，神经网络的输入、输出数据，以及网络中的参数均采用 Tensor 数据结构

b. 属性

i. shape

形状是 Tensor 的一个重要的基础属性，可以通过 `Tensor.shape` 查看一个 Tensor 的形状，以下为相关概念：

- shape：描述了 Tensor 每个维度上元素的数量。
- ndim：Tensor 的维度数量，例如向量的维度为 1，矩阵的维度为 2，Tensor 可以有任意数量的维度。
- axis 或者 dimension：Tensor 的轴，即某个特定的维度。
- size：Tensor 中全部元素的个数。

ii. reshape

`paddle.reshape` 接口来改变 Tensor 的 shape，但并不改变 Tensor 的 size 和其中的元素数据。

- **1** 表示这个维度的值是从 Tensor 的元素总数和剩余维度自动推断出来的。因此，有且只有一个维度可以被设置为 -1。
- **0** 表示该维度的元素数量与原值相同，因此 shape 中 0 的索引值必须小于 Tensor 的维度（索引值从 0 开始计，如第 1 维的索引值是 0，第二维的索引值是 1）。

origin:[3,2,5] reshape:[-1,5] actual: [6,5]# 转换为 2 维，固定一个维度 5，另一个维度根据元素总数推断出来是 $30 \div 5 = 6$
 origin:[3, 2, 5] reshape:[0, -1] actual: [3, 10]# reshape:[0, -1]中 0 的索引值为 0，按照规则，转换后第 0 维的元素数量与原始 Tensor 第 0 维的元素数量相同，为 3；第 1 维的元素数量根据元素总值计算得出为 $30 \div 3 = 10$ 。

除了 paddle.reshape 可重置 Tensor 的形状，还可通过如下方法改变 shape：

- paddle.squeeze，可实现 Tensor 的降维操作，即把 Tensor 中尺寸为 1 的维度删除。
- paddle.unsqueeze，可实现 Tensor 的升维操作，即向 Tensor 中某个位置插入尺寸为 1 的维度。
- paddle.flatten，将 Tensor 的数据在指定的连续维度上展平。
- paddle.transpose，对 Tensor 的数据进行重排。

iii. stop_gradient

Tensor 的 stop_gradient 属性

stop_gradient 表示是否停止计算梯度，默认值为 True，表示停止计算梯度，梯度不再回传。在设计网络时，如不需要对某些参数进行训练更新，可以将参数的 stop_gradient 设置为 True。

iv. dtype

Tensor 的数据类型 dtype 可以通过 Tensor.dtype 查看，支持类型包括：bool、float16、float32、float64、uint8、int8、int16、int32、int64、complex64、complex128"

对于 Python 整型数据，默认会创建 int64 型 Tensor；

对于 Python 浮点型数据，默认会创建 float32 型 Tensor，并且可以通过 `paddle.set_default_dtype` 来调整浮点型数据的默认类型。

通过 Numpy 数组或其他 Tensor 创建的 Tensor，则与其原来的数据类型保持相同。

"飞桨框架提供了 `paddle.cast` 接口来改变 Tensor 的 dtype：

```
float32_Tensor = paddle.to_tensor(1.0)
```

```
float64_Tensor = paddle.cast(float32_Tensor, dtype='float64')
```

c. 操作

i. API 有原位 (Inplace) 操作和非原位操作之分

原位操作即在原 Tensor 上保存操作结果，输出 Tensor 将与输入 Tensor 共享数据，并且没有 Tensor 数据拷贝的过程。非原位操作则不会修改原 Tensor，而是返回一个新的 Tensor。通过 API 名称区分两者，如 `paddle.reshape` 是非原位操作，`paddle.reshape_` 是原位操作。

ii. API 调用有两种方法：

```
print(paddle.add(x,y), "\n")# 方法一：使用 Paddle 的 API
print(x.add(y), "\n")# 方法二：使用 tensor 类成员函数
```

d. 创建

i. 指定数据创建

可使用 `paddle.to_tensor` 创建任意维度的 Tensor

1. Tensor 必须形如矩形，即在任何一个维度上，元素的数量必须相等，否则会抛出异常
2. 飞桨也支持将 Tensor 转换为 Python 序列数据，可通过 `paddle.tolist` 实现，飞桨实际的转换处理过程是 **Python 序列 ↔ Numpy 数组 ↔ Tensor**。
3. 基于给定数据创建 Tensor 时，飞桨是通过拷贝方式创建，与原始数据不共享内存。

```
ndim_3_Tensor = paddle.to_tensor([[[[1, 2, 3, 4, 5],  
                                     [6, 7, 8, 9, 10]],  
                                   [[11, 12, 13, 14, 15],  
                                   [16, 17, 18, 19, 20]]])
```

ii. 指定形状创建

如果要创建一个指定形状的 Tensor，可以使用 `paddle.zeros`、`paddle.ones`、`paddle.full` 实现。

```
paddle.zeros([m, n]) # 创建数据全为 0，形状为 [m, n] 的Tensor  
paddle.ones([m, n]) # 创建数据全为 1，形状为 [m, n] 的Tensor  
paddle.full([m, n], 10) # 创建数据全为 10，形状为 [m, n] 的Tensor
```

iii. 指定区间创建

如果要在指定区间内创建 Tensor，可以使用 `paddle.arange`、`paddle.linspace` 实现

```
paddle.arange(start, end, step)# 创建以步长 step 均匀分隔区间[start, end)的 Tensor  
paddle.linspace(start, stop, num)# 创建以元素个数 num 均匀分隔区间[start, stop) 的 Tensor
```

iv. 自动创建

实际在飞桨框架中有一些 API 封装了 Tensor 创建的操作，从而无需用户手动创建 Tensor。

- 例如 `paddle.io.DataLoader` 能够基于原始 Dataset，返回读取 Dataset 数据的迭代器，迭代器返回的数据中的每个元素都是一个 Tensor。
- 另外在一些高层 API，如 `paddle.Model.fit`、`paddle.Model.predict`，如果传入的数据不是 Tensor，会自动转为 Tensor 再进行模型训练或推理。

说明：

`paddle.Model.fit`、`paddle.Model.predict` 等高层 API 支持传入 Dataset 或 DataLoader，如果传入的是 Dataset，那么会用 DataLoader 封装转为 Tensor 数据；如果传入的是 DataLoader，则

直接从 DataLoader 迭代读取 Tensor 数据送入模型训练或推理。因此即使没有写将数据转为 Tensor 的代码，也能正常执行，提升了编程效率和容错性。

v. 其他

除了以上指定数据、形状、区间创建 Tensor 的方法，飞桨还支持如下类似的创建方式

- **创建一个空 Tensor**，即根据 shape 和 dtype 创建尚未初始化元素值的 Tensor，可通过 `paddle.empty` 实现。
- **创建一个与其他 Tensor 具有相同 shape 与 dtype 的 Tensor**，可通过 `paddle.ones_like`、`paddle.zeros_like`、`paddle.full_like`、`paddle.empty_like` 实现。
- **拷贝并创建一个与其他 Tensor 完全相同的 Tensor**，可通过 `paddle.clone` 实现。
- **创建一个满足特定分布的 Tensor**，如 `paddle.rand`、`paddle.randn`、`paddle.randint` 等。
- **通过设置随机种子创建 Tensor**，可每次生成相同元素值的随机数 Tensor，可通过 `paddle.seed` 和 `paddle.rand` 组合实现。

vi. 图像数据创建

在常见深度学习任务中，数据样本可能是图片（image）数据和对应的标签均需要创建为 Tensor。

- 对于图像场景，可使用 `paddle.vision.transforms.ToTensor` 直接将 PIL.Image 格式的数据转为 Tensor，使用 `paddle.to_tensor` 将图像的标签（Label，通常是 Python 或 Numpy 格式的数据）转为 Tensor。

ViT课程学习

- 技巧
 - who am i
 - where am i
 - what should i do
- Github重温
 - 协作
 - fork
 - git clone
 - git checkout -b Name
 - git status
 - git add
 - git commit -m
 - git push origin Name
 - pull request
- 复现resnet
 - 方法
 - download paper
 - according to map 复刻实现
 - 抽象总结为表格

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
		3×3 max pool, stride 2				
conv2.x	56×56	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3.x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4.x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5.x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

- epoch
 - 所有batch

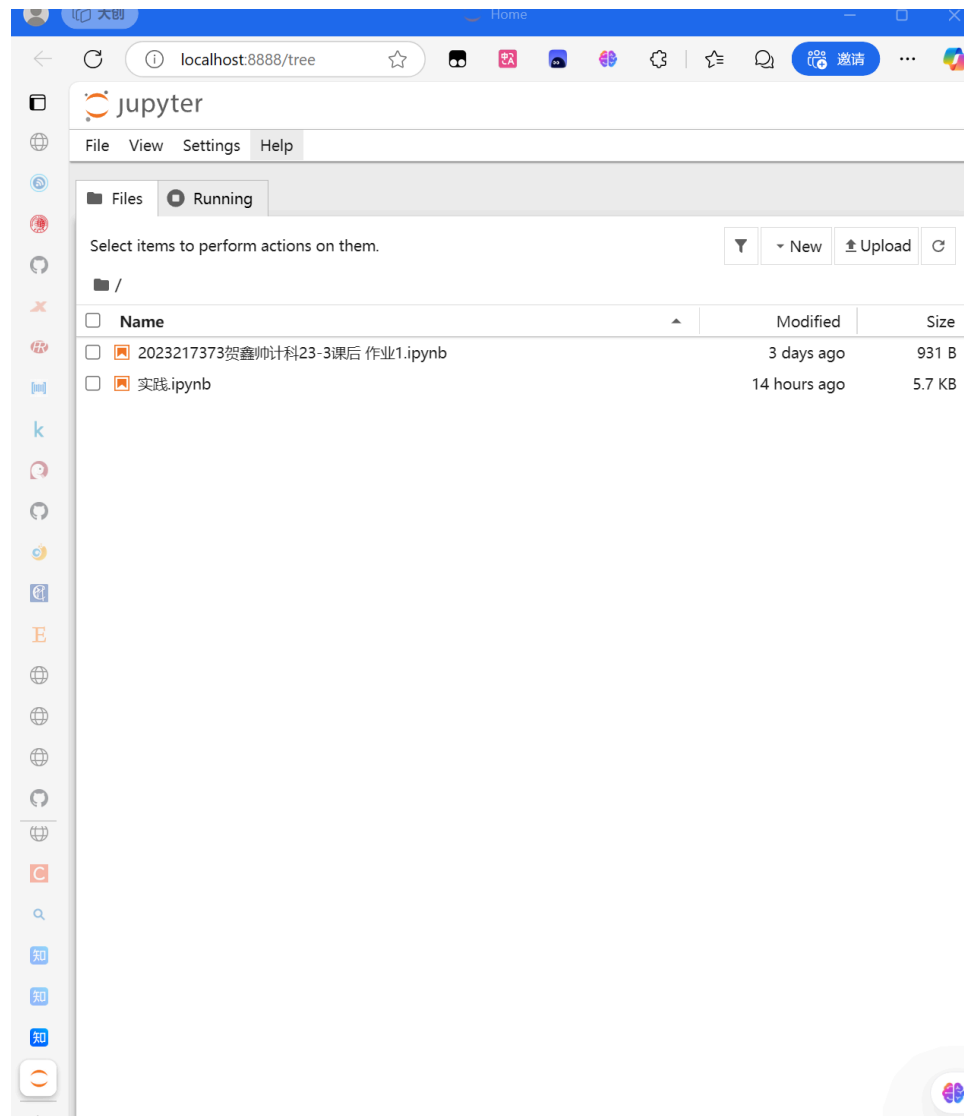
- 前向
- 反向
- 图像
 - pixel
 - ndarray 32*32*3
 - to_tensor→32*32*3
 - 常见操作
 - reshape
 - expand
- Transformer
 - 编码encoder
 - Msa
 - ffn
 - Add
 - Layer Normal
 - 解码
 - Vit
 - image tokens (to sequence)
 - 分块
 - reshape
 - reshape展平
 - Linear Projection
 - 多个encoder
 - head

相关环境学习

- jupyter notebook
 - 更改存放位置
 - 获取配置文件位置

```
jupyter notebook --generate-config
```

- 修改notebook_dir



- 启动
 - jupyter notebook
 - --port <port_number>
 - --no-browser
- 语法
 - shell命令
 - !shell命令
 -
 - 加载运行外部代码

- 网站源代码

```
%load URL
```

- 本地文件

```
%load Python文件的绝对路径
```

- 运行

```
%run Python文件的绝对路径
```

- 退出

- ctrl+c

- 拓展安装

- 链接conda

```
conda install nb_conda
```

```
canda remove nb_conda
```

- 添加conda内核

- `conda activate your_environment_name`
- `conda install ipykernel`
- `python -m ipykernel install --user --name=your_environment_name`

- 安装插件

```
conda install -c conda-forge jupyter_contrib_nbextensions
```

- conda管理环境

- 创建新环境

```
conda create --name <env_name> <package_names>
```

- 复制环境

```
conda create --name <new_env_name> --clone <copied_env_name>
```

- 删除环境

```
conda remove --name <env_name> --all
```

- 切换环境

```
activate <env_name>
```

- 退出环境

```
deactivate
```

- 显示环境

```
conda env list
```

- 包管理

- 查看已安装

```
conda list
```

- 搜索可安装

```
conda search --full-name <package_full_name>
```

```
conda search <text>
```

- 安装包

```
conda install --name <env_name> <package_name>
```

```
conda install <package_name>
```

```
pip install <package_name>
```

- 卸载包

```
conda remove <package_name>
```

实践

- 配置paddlepaddle
 - 安装

```

PS C:\Users\Iron> cd d:
PS D:\> cd python
PS D:\python> python
Python 3.12.7 (tags/v3.12.7:0b05ead, Oct 1 2024, 03:06:41) [MSC v.1941 6
4 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import paddle
信息：用提供的模式无法找到文件。
D:\Python\Lib\site-packages\paddle\utils\cpp_extension\extension_utils.py
:711: UserWarning: No ccache found. Please be aware that recompiling all
source files may be required. You can download and install ccache from: h
ttps://github.com/ccache/ccache/blob/master/doc/INSTALL.md
  warnings.warn(warning_message)
>>> paddle.utils.run_check()
Running verify PaddlePaddle program ...
I0324 22:00:47.477174 18196 pir_interpreter.cc:1508] New Executor is Runn
ing ...
W0324 22:00:47.477174 18196 gpu_resources.cc:119] Please NOTE: device: 0,
GPU Compute Capability: 8.9, Driver API Version: 12.7, Runtime API Versi
on: 12.3
W0324 22:00:47.478175 18196 gpu_resources.cc:164] device: 0, cuDNN Versio
n: 9.1.
I0324 22:00:47.480175 18196 pir_interpreter.cc:1531] pir interpreter is r
unning by multi-thread mode ...
PaddlePaddle works well on 1 GPU.
PaddlePaddle is installed successfully! Let's start deep learning with Pa
dplePaddle now.
>>> |

```

经验总结

- 学习技术文档时，用notebook边学边敲代码是个非常不错的体验

想法

- 部署
 - 服务器
 - 加密
 - 端侧
 - 性能优化

下一步

- 学习paddlepaddle框架
- 简单学习命令行
- 学习vit
- 复刻实现swin_transformer

